

Data Structures – Unit 07

Bucharest University of Economic Studies

Hash Tables



Agenda

- Definition
- Characteristics
- Representation
- Implementation
- Applications
- Metrics



Hash Table - Definition

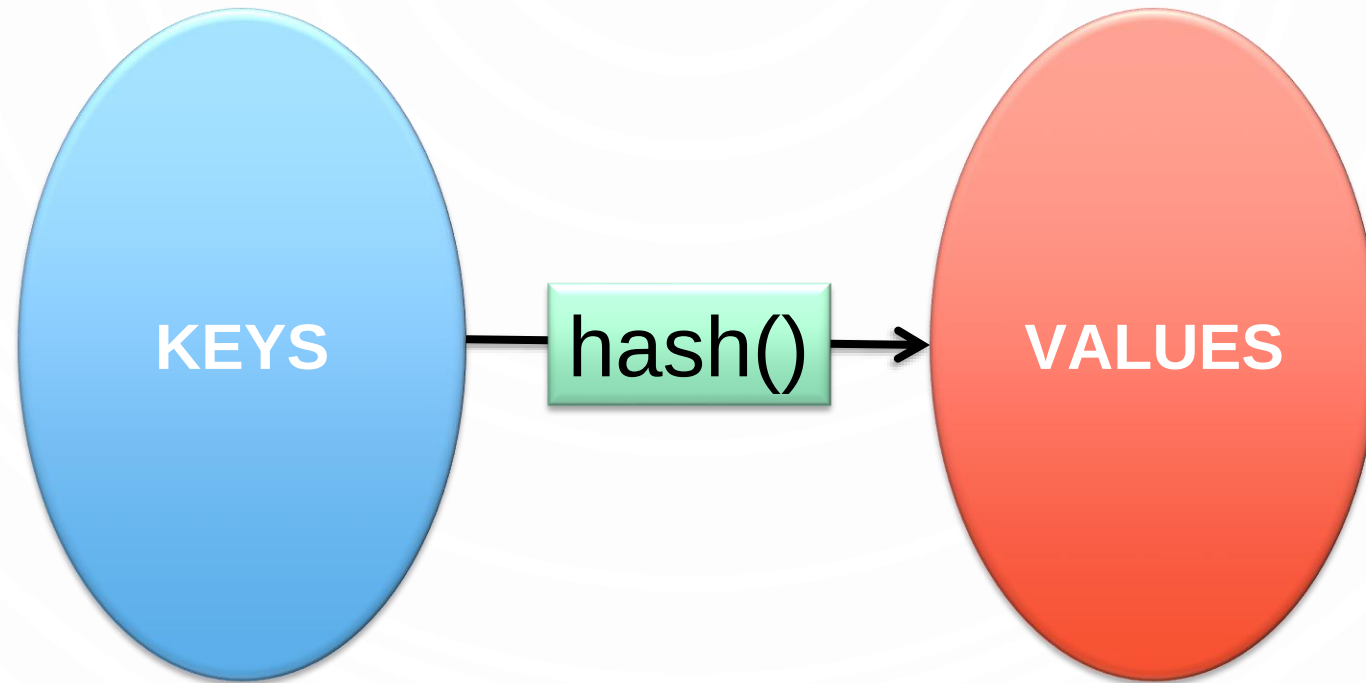
A **hash table** is a data structure which puts together two sets, binding them uniquely by using a hash function.

These two sets are composed of:

- unique keys - indices;
- values – data stored at an index location;



Hash Table



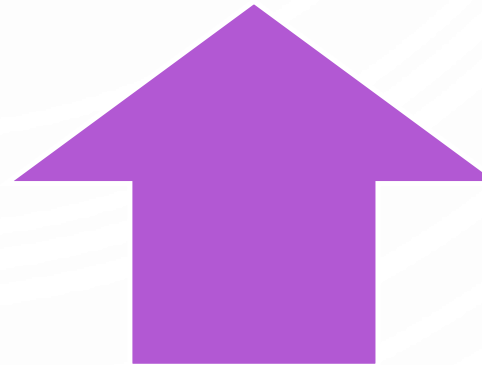
Hash Table - Objective



Space



Speed



Direct access table - arrays

Advantages:

- **Searching in $O(1)$ Time:** Direct address tables use arrays which are random access data structures, so, the key values (which are also the index of the array) can be easily used to search the records in $O(1)$ time.
- **Insertion in $O(1)$ Time:** We can easily insert an element in an array in $O(1)$ time. The same thing applies for a direct address table.
- **Deletion in $O(1)$ Time:** Deletion of an element takes $O(1)$ time in an array. Similarly, to delete an element in a direct address table we need $O(1)$ time.



Direct access table

Limitations:

- We need to know the maximum value from the key set;
 - thus, it is only useful if the maximum value is very small.
- Uses too much memory if there is a significant difference between total number of records and the maximum value.
 - few elements with very high values for keys;



Direct access tables

Disadvantages of using direct access tables:

- Total amount of memory used:

$$mem_size = \max_i(key_i) * sizeof(element)$$

- Doesn't consider which is the actual number of used elements:
 - worst case scenario: few elements with high key values;
- The type of the search key:
 - numeric type – must be an index of an array;
 - the key must be unique;
 - must be densely stacked and not sparse;



Solution: Hash Table

HT data structure is used to improve the search operation:

- Each value has a unique key associated with it;
- The key helps determine the value from the structure with almost constant complexity, $O(1)$;

The implementation of HT uses one of the following:

- A dynamic or static array of keys and elements;
- A list with values for collisions;



Characteristics

- The total number of keys must be previously known in order to correctly size a one-dimensional array;
- If the array is undersized then many elements will be crowded into a small area, leading to the creation of clusters;
- If the array is too big, then memory will be wasted;
- Minimizes data search operations by using direct access;



Characteristics

- Is a data structure used for storing and fast searches;
- The index or the key at which an element can be found is given by the hash function;
- An efficient way of managing memory;
- Hash functions have different complexity levels influencing how items are spread across the entire structure;

$$\text{hash}(k) = k \bmod m$$



Characteristics

Advantages of using hash tables:

- Using alphanumeric keys for the search operation;
- The hash function makes the connection between the alphanumeric key and the index of the element;
- Uses the index computed by the hash function to directly access the elements of the structure;



Characteristics

Hash Tables disadvantage:

- Need extra processing for computing the hash function;
- The concept of collision which states that two different values can have the same hashed key: X_h, Y_h , where $X_h \neq Y_h$, have identical hash keys $\text{hash}(X_h) = \text{hash}(Y_h)$;
- Collision avoidance needs extra processing such as: chaining, re-hashing, linear probing, quadratic probing or overflow area.



Characteristics

The hash key must be a deterministic function;

It should evenly distribute keys across buckets;

The size of the key set is smaller compared with the size of the elements that need to be mapped.



Key classification:

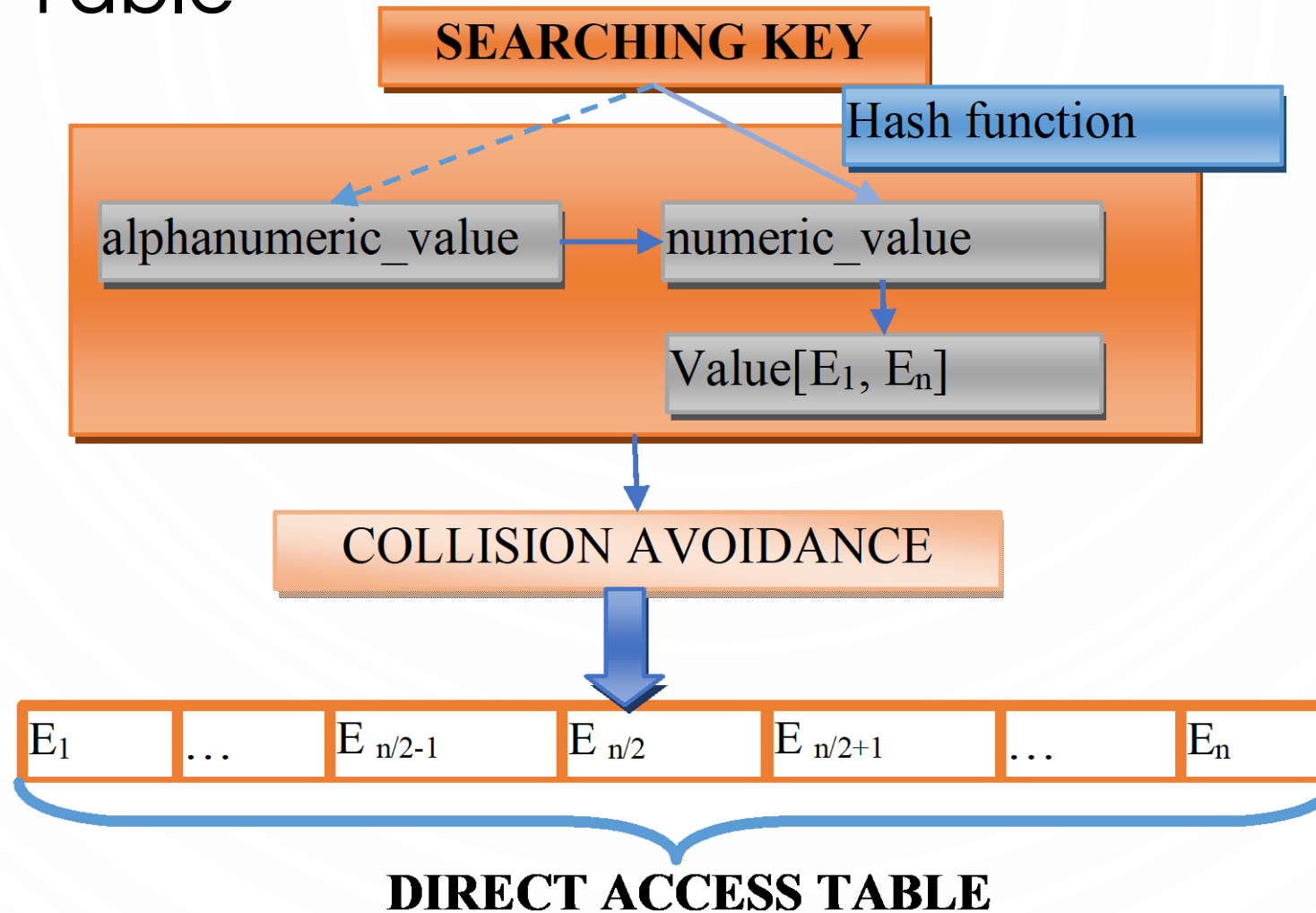
Numeric keys: easy to process;

Alphanumeric keys: array of characters or strings;

Compound keys: based on multiple fields;



Hash Table



Hash Table

The loading factor of a hash table is defined as:

$$G = \frac{n}{m}$$

n – the number of elements stored in the structure;

m – the total number of positions or hash values available;



Hash Table

- Based on how data is loaded into a hash table we deduce:
 - uniform hashing – any element is equally likely to hash into any of the hash table slots, independently of the other elements;
 - poor hashing – takes two or more different elements to the same bucket in the hash table, worst case scenario is the one when all the elements are mapped into the same hash value;



Hash Functions

The main objective of the hash function is to spread as uniformly and randomly as possible, the elements into the hash table.



Hash Functions

- Processes the key of each record;
- Identifies data inside the hash table;
- There's no generic hash function;
- Choosing hash function is made based on the characteristics of the value set;



Mathematical models

Processing the ASCII codes of alphanumeric characters:

$$\text{hashValue} = \text{keyString}[0] \% 255$$

hashValue: the hashed value of the key;

keyString[0]: the first character of the string that represents the searching key;



Hash Functions

hashValue : is a low complexity model used for managing a small number of elements;

The model generates lots of collisions because many strings could have the first element identical.



Hash Functions

Refining the model: using more than one character in the computation of the hash value (the first and the last):

$$\text{hashValue} = (\text{keyString}[0] + \text{keyString}[\text{length}_{\text{keyString}}]) \% n$$

hashValue: the hashed value of the key;

keyString: the searching key;

$\text{length}_{\text{keyString}}$: the number of characters of the string

n: hash table dimension



Hash Functions

In order not to reduce the size of the hash table to only 255 elements, a high enough prime number can be used;

Processing the entire string reduces the number of collisions induced by the hash function;

$$val_{hash} = \left(\sum_{i=1}^{keyLength} ASCII(key[i]) \right) \% n$$



Hash Functions

Hash function for a string type key:

$$\text{hash}(S) = s[0]*31^{n-1} + s[1]*31^{n-2} + \dots + s[n-2]*31 + s[n-1]$$

$s[i]$ - ASCII code; n – the size of the string;

$\text{hash}(\text{"hello"}) =$

$$= 104*31^4 + 101*31^3 + 108*31^2 + 108*31^1 + 111*31^0 =$$

$$= 99162322.$$

The value of the hash function can be rescaled in order to identify the index of data that needs to be accessed;



Mathematical models

Modulo operation: low complexity and easiness in implementation with low complexity; searching key is mapped to a numerical value found in the set $[0; n-1]$, n being the size of the hash table:

$$\text{hashTable_index} = \text{keyValue} \% n$$

hashTable_index: the hash of the key;

keyValue: the value of the numeric key;

n : a prime number which is less or equal to the size of the hash table; generalValue: $(4*i+3)$ with $i = 0, 1, 2, 3, \dots$



Mathematical models

The multiplication with a random real number from $[0;1)$ and the processing of the decimal part; by multiplying the result with the size n of the hash table give us access to indexes between $[0; n-1]$;

$$\text{hashValue} = \{n[(\text{keyValue} * \text{random}_{[0;1)}) \bmod 1]\}$$

hashValue: the hashed value of the key;

keyValue: the value of the searching key;

$\text{random}_{[0;1)}$: a random number between $[0;1)$

n : the size of the hash table



Hash Table - Hash Functions

Collision avoidance

Separate
chaining

Open
addressing



Avoiding collisions

- Methods of managing elements in a hash table in which two different values generates two identical hashes:
- Separate chaining: using an array of linear lists;
- Open addressing: re-hashing , linear probing, quadratic probing, overflow area.

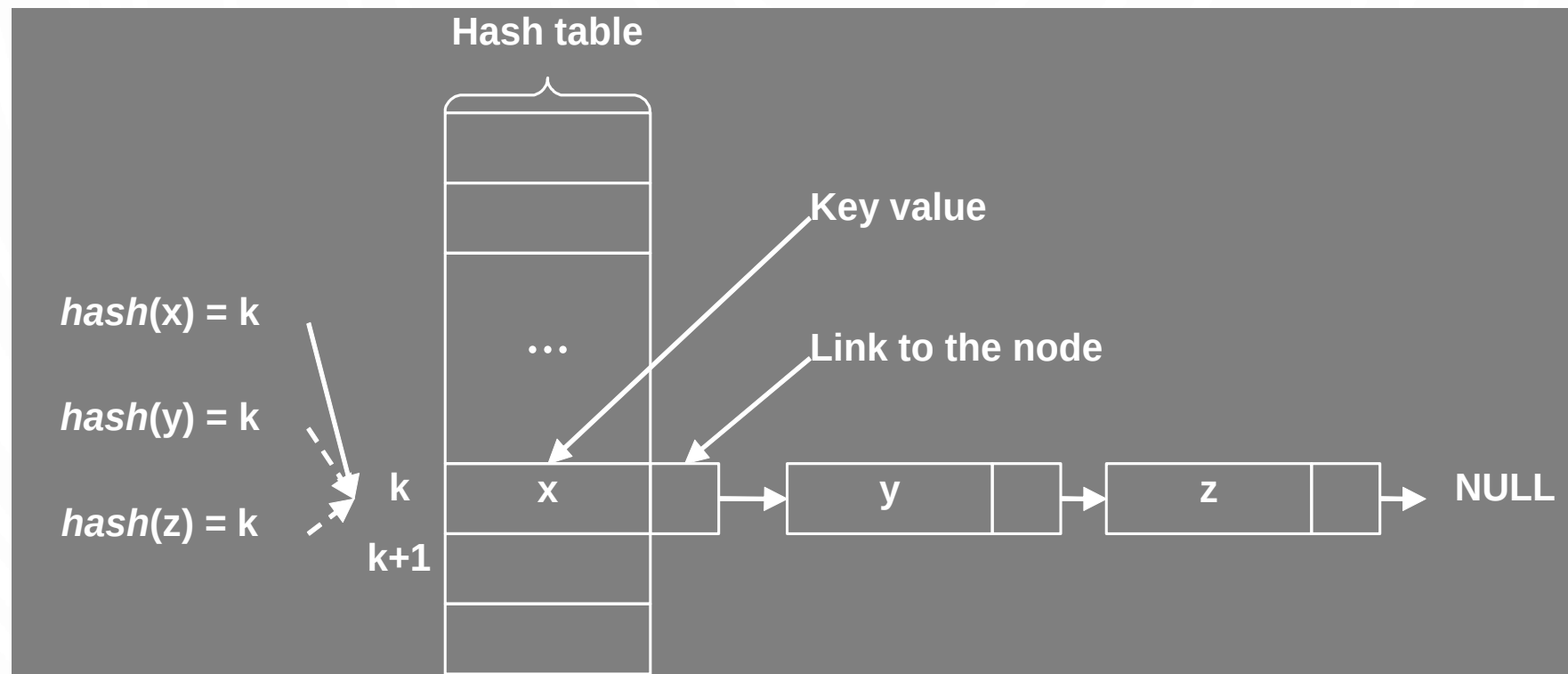


Collisions - Chaining

- Uses an array of buckets;
- Each slot has its own linked list of collisions;
- All the elements from a list have the same hash values;
- Searching an element mean to identify the correct bucket based on the hash value after which the list needs to be crossed;



Hash Table - Collisions

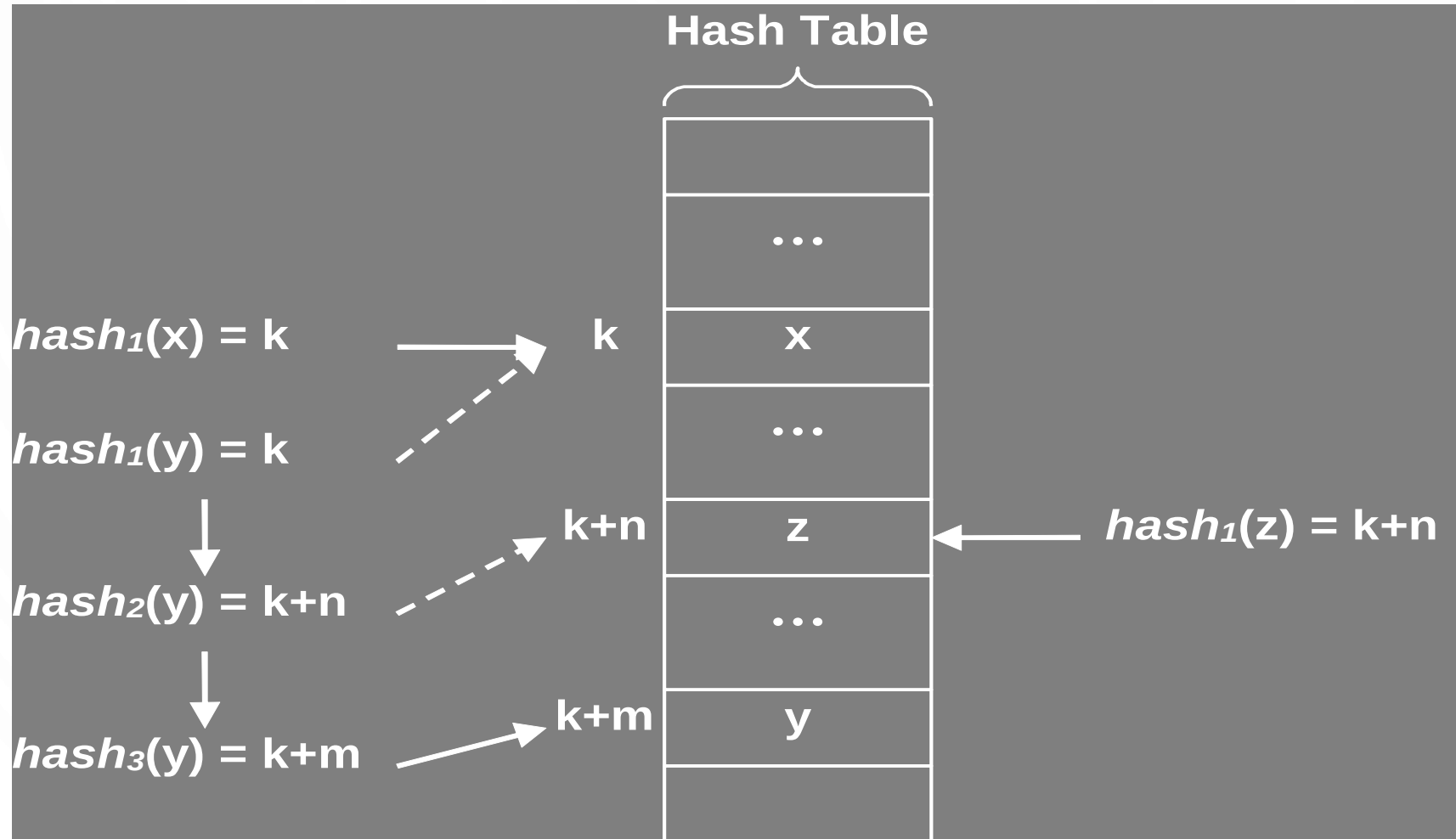


Collisions – Cascade Hashing

- Applying multiple hashing functions until the value obtained is a free position from the hash table;
- Each step of the searching process computes a new hash value of the key until an empty position is found or until there is no possibility for computing another hash.



Hash Table - Collisions

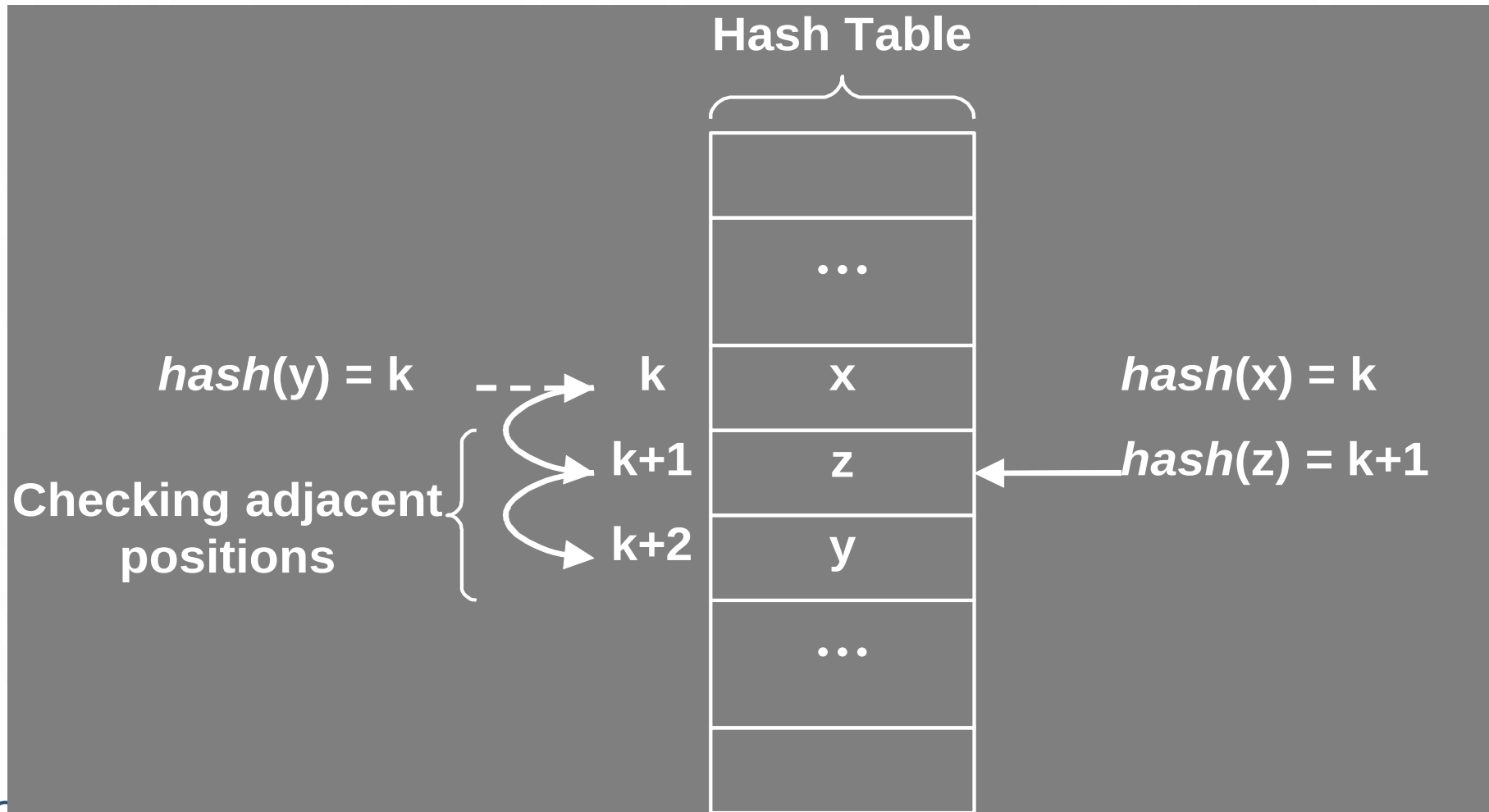


Collisions – Linear probing

- Sequential searches of the first free bucket in which the new element can be inserted, to the left or right of the position pointed by the hash value (collision place);
- Search in a hash table implemented with linear probing: all adjacent elements to the position referred by the hash value must be checked;
- Disadvantage: if the number of collisions is high than clusters can develop; the probability of having collisions not only for same values of hash but also for adjacent values;



Hash Table - Collisions



Collisions – Quadratic probing

- Probing for free slots inside the hash table by using a step; In linear probing the step is ± 1 ;
- Reduces the probability of having collisions for the adjacent hash values and spreads the elements uniformly in the structure;
- Reduces the risk of clusters for repeated collisions



Collisions – Quadratic probing

Calculating the next free bucket from the hash table:

$$\text{nextPosition} = \text{hash}(X) + c * i^2$$

nextPosition: the new position calculated based on the hash value of the key and a variable for avoiding cluster formations;

X: the searching key associated with the element;

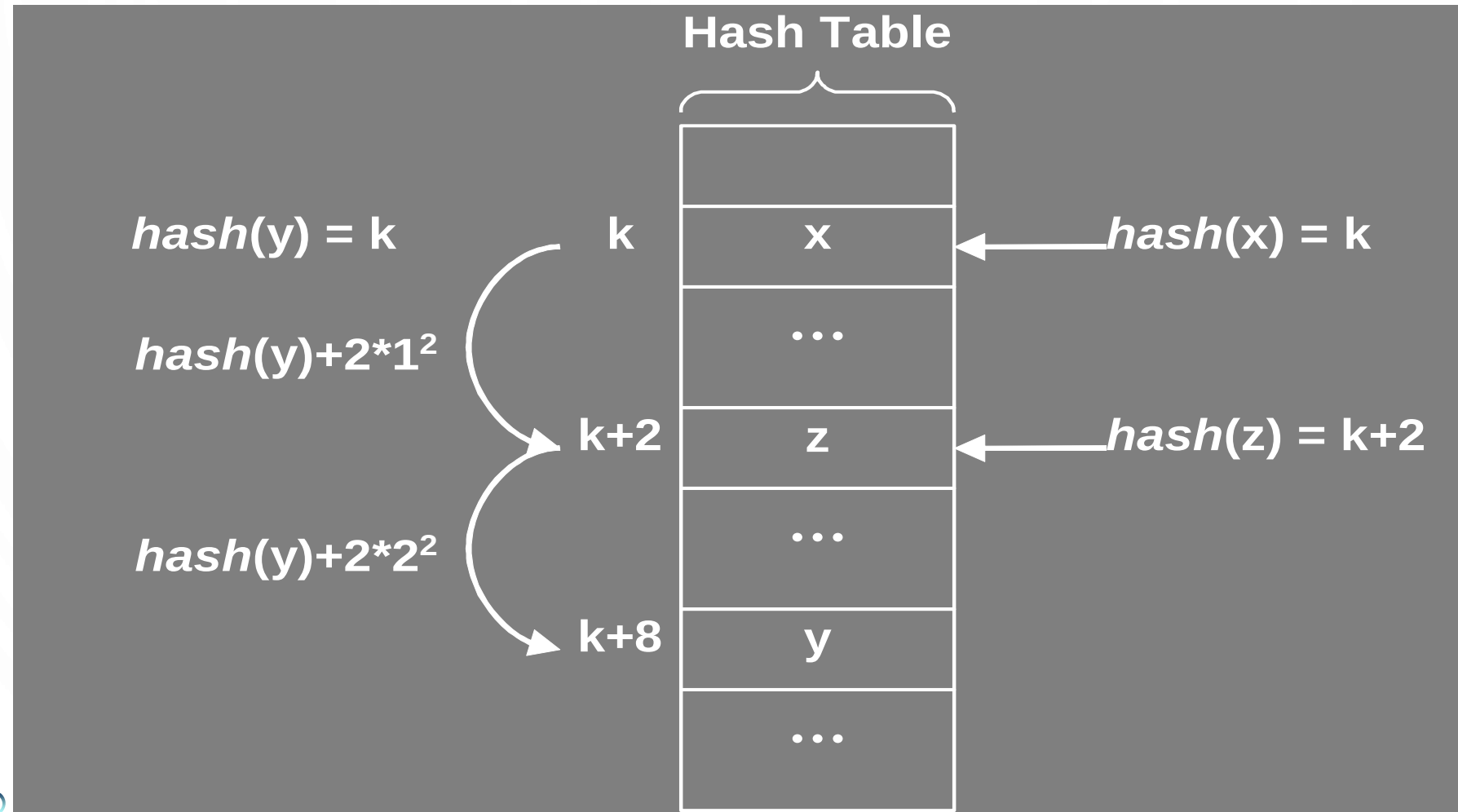
hash(X): the hashed value of the searching key;

c: a constant value {1, 2, 4, ...};

i: number of rehash operations or the number of checked positions;



Hash Table - Collisions

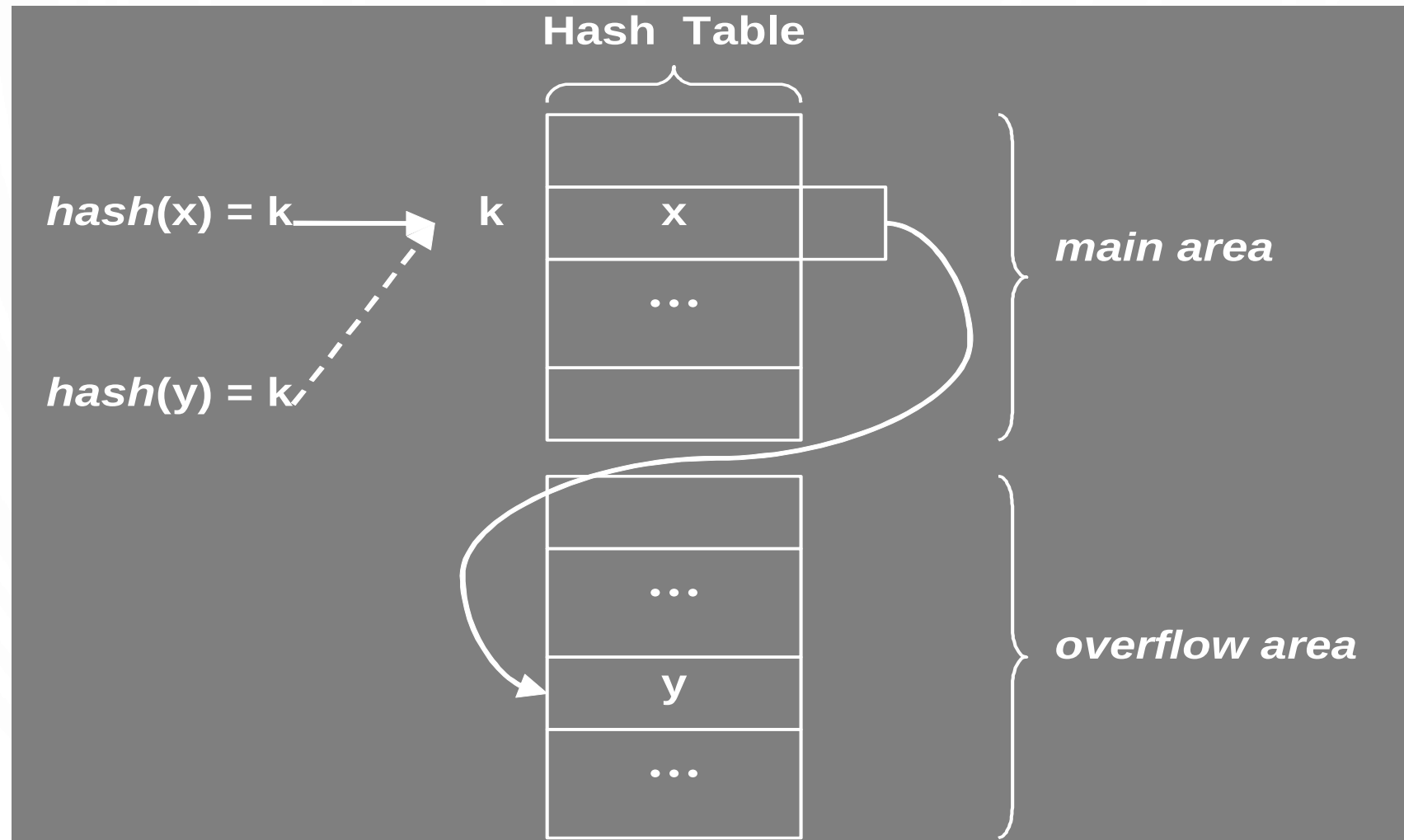


Collisions – Overflow area

- Splits the hash table in two regions:
 - Primary area used for storing the initial elements;
 - Secondary area used for storing the collisions;
- When searching for an element a pointer is used for referring to the collision area locations;
- The access to the secondary area is made by using a pointer stored in the primary area;
- The search time operation is almost constant $O(1)$, much better than other types of implementations;



Hash Table - Collisions



Hash Table - Applications

- Database management systems (DBMS's)
 - sequential access vs. random access
- Tables that store symbols
 - compilers
- Data dictionaries



Hash Table

The probability of a collision increasing directly with the loading factor of the hash table.

The hash functions that have a low degree of complexity are creating collisions more often than others with a high degree of complexity.



Efficiency

- Loading factor:
- $\lambda = \frac{n}{m} < 0.7$
 - n – no. of inserted keys;
 - m – no. of available positions, no. of slots;
- Rehashing the hash table
 - for increasing the capacity
 - Improving the complexity



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

